Kubernetes 网关流量管理：Ingress 与 Gateway API

Original rifewang 系统架构师Go 2024年08月31日 06:57

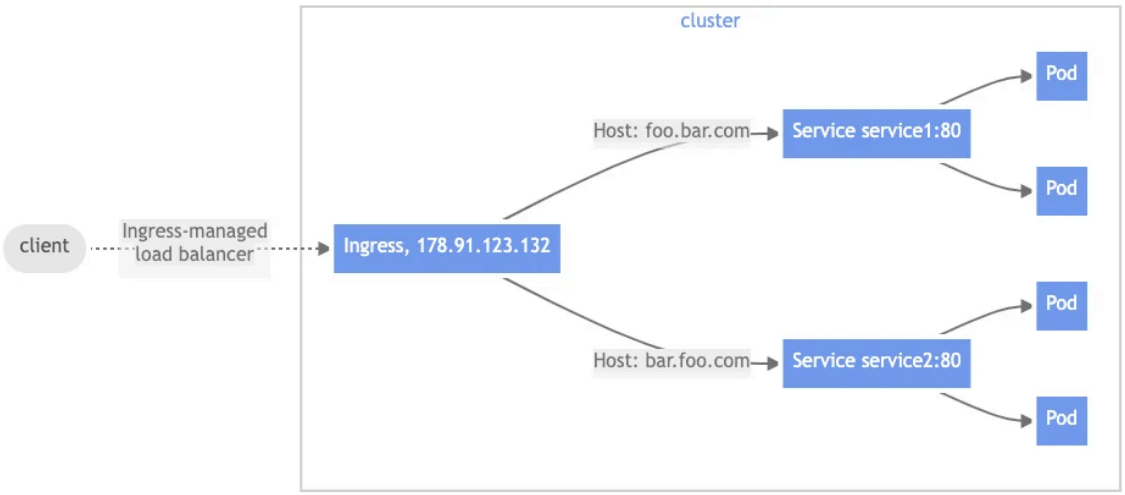
引言

随着 Kubernetes 在云原生领域的广泛使用，流量管理成为了至关重要的一环。为了有效地管理从外部流入集群的流量，Kubernetes 提供了多种解决方案，其中最常见的是 Ingress 和新兴的 Gateway API 。

Ingress

随着微服务架构的发展，服务的数量和复杂性不断增加，如何高效、安全地管理进入集群的外部流量成为了一个亟待解决的问题。Kubernetes 的 Ingress 资源应运而生，它旨在提供一种简单的方式来配置 HTTP 或 HTTPS 流量的路由，并且可以通过反向代理将外部流量分发到集群内部的服务。

例如，将不同的 host 分发到不同的服务：




```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
  - host: foo.bar.com
```

```
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: service1
            port:
              number: 80
  - host: bar.foo.com
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: service2
            port:
              number: 80
```

尽管 Ingress 为流量管理提供了方便的入口配置，但它也存在一些致命缺陷：

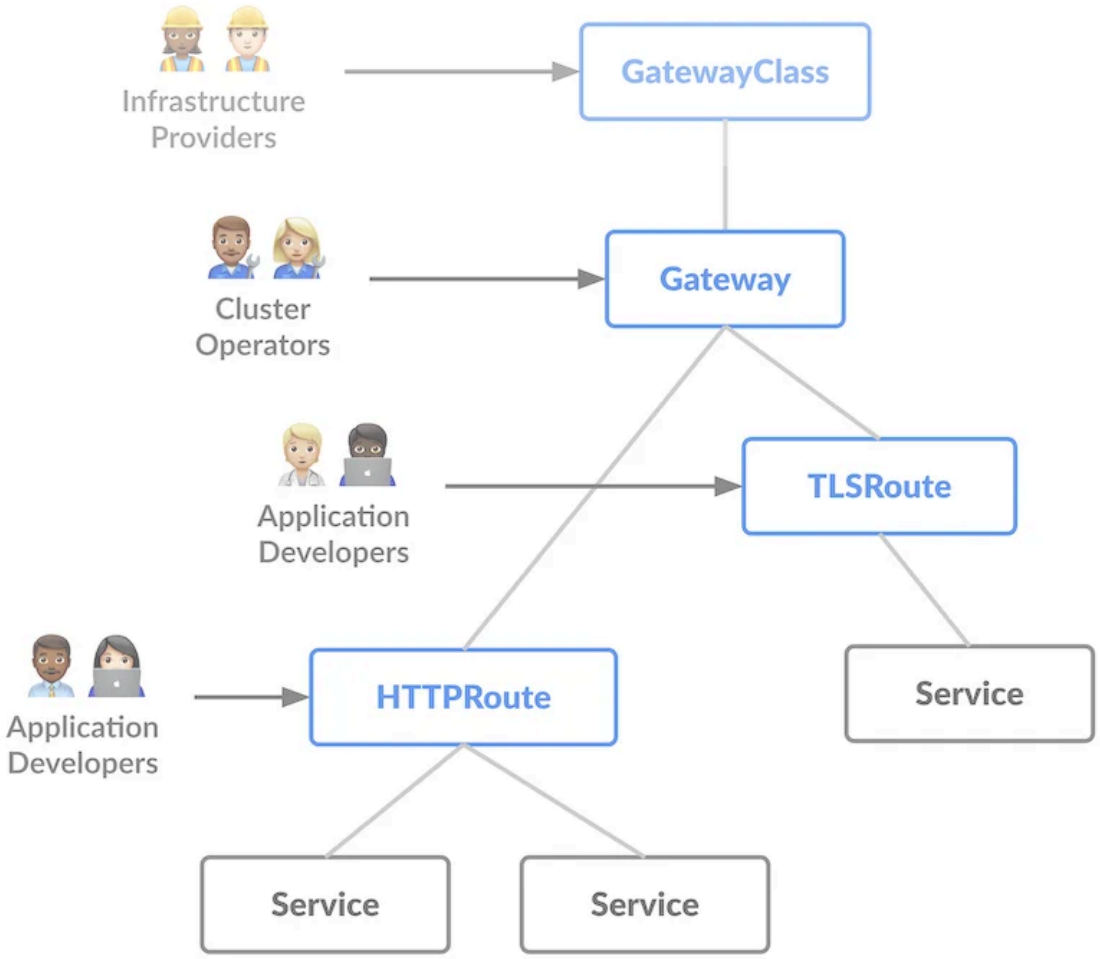
- 缺乏灵活性： Ingress 往往只支持 L7 HTTP(S) 协议，缺乏对其他协议（如 L4 TCP/UDP）的支持。
- 供应商依赖：不同的 Ingress 控制器具有不同的实现，功能差异较大，容易导致锁定某个特定供应商的实现（比如在不同的网关里配置不同的 annotations ）。
- 扩展性有限：随着集群规模的扩大和复杂化，单一的 Ingress 资源可能难以满足大规模集群的需求。

Gateway API

为了弥补 Ingress 的局限性，社区引入了 Gateway API 。它不仅提供了更强大的流量路由功能，还提升了对多种协议、跨团队协作、以及多供应商兼容性的支持。 Gateway API 的目标是将流量管理从服务网格、Ingress 控制器等解耦出来，提供一个标准化的 API 用于控制流量。

Gateway API 的优点有：

- 统一化标准化：由于定义了一套标准化的 API，即使替换了底层使用的网关，上层仍然可以保持无感的，这也避免了网关的供应商依赖。
- 更多协议支持： Gateway API 支持 L4 ~ L7 多种协议，包括 HTTP、gRPC、TCP、UDP 等。
- 分层设计与职责分离： Gateway API 引入了 GatewayClass 、 Gateway 和 Route 等资源，不同的角色管理不同的资源。



GatewayClass

GatewayClass 由基础设施人员管理，用来描述底层的网关实现（例如，你可以有不同的 GatewayClass 来分别代表使用 Nginx、Istio、或者其他负载均衡器的网关）。

例如，以下定义了一个 Nginx 的网关实现：

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  labels:
    app.kubernetes.io/instance: nginx-gateway
    app.kubernetes.io/name: nginx-gateway
    app.kubernetes.io/version: 1.4.0
  name: nginx
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
```

Gateway

Gateway 由集群管理员负责，是流量入口的具体定义，用来代表一个物理或虚拟的流量入口点。Gateway 资源会引用一个 GatewayClass 来确定网关的实现方式，并包含一组监听器（listeners），这些监听器定义了网关监听的端口和协议。

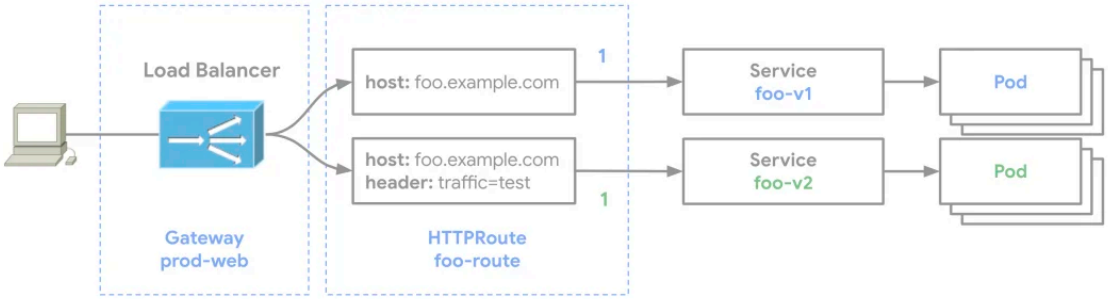
例如，以下定义了一个网关，监听 80 端口的 HTTP 入站流量，底层使用 nginx 网关实现：

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-web
spec:
  gatewayClassName: nginx
  listeners:
  - protocol: HTTP
    port: 80
    name: prod-web-gw
```

Route

Route 定义路由规则，针对不同的协议类型使用不同的资源，如 HTTPRoute 、 TLSRoute 、 TCPRoute 、 UDPRoute 、 GRPCRoute 等。

例如，使用 HTTPRoute 根据 HTTP 请求头分发到不同的服务：




```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: foo-route
  labels:
    gateway: prod-web-gw
spec:
  hostnames:
  - foo.example.com
  rules:
  - backendRefs:
```

```
      - name: foo-v1
        port: 8080
    - matches:
      - headers:
        - name: traffic
          value: test
      backendRefs:
      - name: foo-v2
        port: 8080
```

合理配置 Route 便可以轻松实现金丝雀发布、蓝绿部署、流量镜像、重定向与请求改写等各种复杂的路由功能。

GAMMA

我们把集群内外方向的流量称为“南北向流量”，把集群内部服务之间的流量称为“东西向流量”。

虽然 Gateway API 最开始的关注点集中在南北向流量上，但是随着 service mesh 服务网格的兴起，人们也希望能将东西向流量一并统一化与标准化。因此 Gateway API 自 v1.1.0 版本起，已将 GAMMA （Gateway API for Mesh Management and Administration）倡议的工作纳入标准渠道，并且有一个专门的子项目来做这件事。

简而言之， Gateway API 正在实现集群内外流量管理的大一统。

总结

Ingress 是否会被 Gateway API 替代？不会， Ingress 自 1.19 版本 GA 后并没有计划弃用。

Ingress 与 Gateway API 都是声明性资源（可以简单理解为配置文件），两者都需要由具体的网关支持工作。

Ingress 提供了较为简单的流量管理，适用于小规模和简单的流量需求，其实现依赖于具体的控制器，因此存在一定的供应商绑定问题。

Gateway API 的标准化设计提供了跨平台的兼容性和一致性，并且支持多协议、多团队协作和更复杂的流量管理需求。

参考资料：

- *https://kubernetes.io/docs/concepts/services-networking/ingress/*
- *https://gateway-api.sigs.k8s.io/*

kubernetes 云原生 35

kubernetes 云原生 · 目录

上一篇
Kubernetes scheduler 概述及自定义调度器

下一篇
Kubernetes 之 kubelet 与 CRI、CNI 的交互过程